Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ ДО ЛАБОРАТОРНОЇ РОБОТИ №5
«Обробка помилок, файли та серіалізація»
з курсу «Професійний Python»

Варіант №8 — «Імпорт та експорт системи бронювання готелю»

Виконав: студент 3 курсу
групи ФеП-32
Івасів Данило

Перевірив: доцент
Куньо Іван Михайлович
Дата: 30.09.2026

Львів — 2026

1. Мета роботи

Сформувати практичні навички створення надійних Python-програм, які коректно працюють із помилками, файловими ресурсами, конфігурацією та структурованими форматами даних. Опанувати механізми exceptions (зокрема exception chaining), context managers, pathlib, streaming I/O та atomic output.

2. Індивідуальний варіант і постановка задачі

Варіант №8 — «Імпорт та експорт системи бронювання готелю». Поля: room_number, room_type, price, status, client.

Завдання:

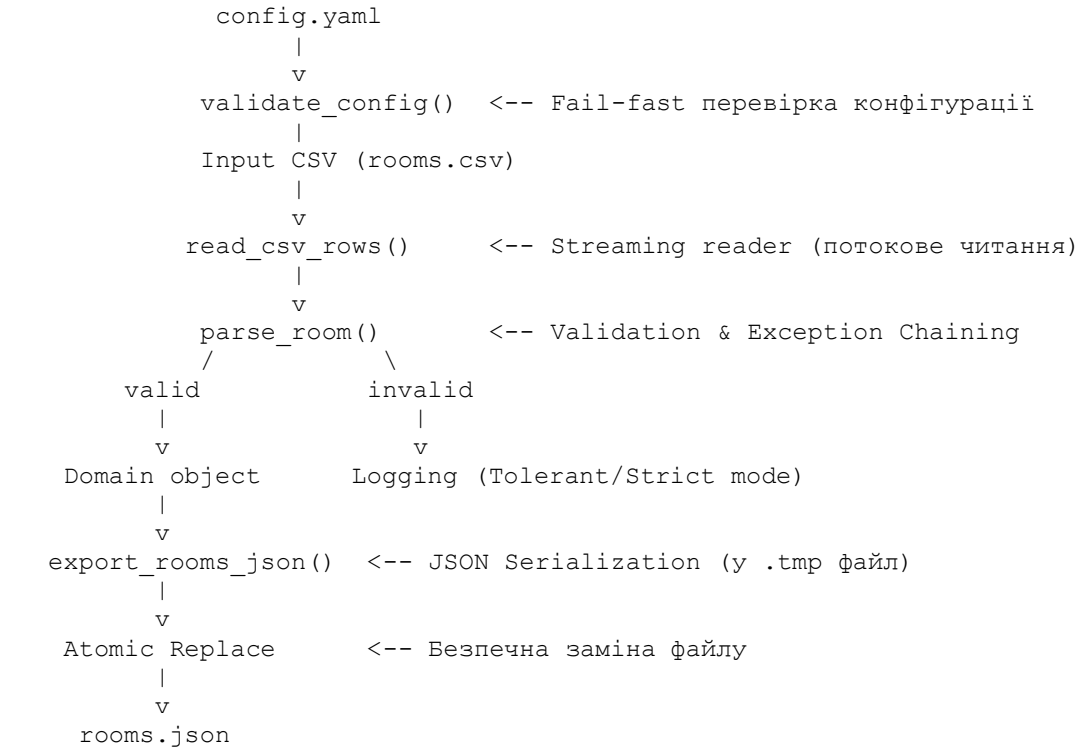
• Перевіряти room number, type, price >= 0, allowed statuses та ім’я клієнта для заброньованих номерів.

• Реалізувати streaming CSV import та JSON export (повинен містити тільки valid records).

• Використати YAML-конфігурацію для задання дозволених типів/статусів та strict/tolerant mode.

• Використати logging для реєстрації некоректних записів та atomic write для збереження output.

3. Схема надійного конвеєра (Reliable Pipeline)



4. Структура проєкту

```
hotel_booking/
├── pyproject.toml
├── config.yaml                 # Конфігурація застосунку
├── README.md
├── .gitignore
```

```
├── data/
│   └── rooms.csv               # Вхідний датасет
├── output/
│   └── rooms.json              # Результат експорту
├── logs/
│   └── application.log         # Файл журналювання
└── src/
    └── reliable_importer/
        ├── __init__.py
        ├── config.py           # Завантаження та валідація YAML
        ├── exceptions.py       # Custom exception hierarchy
        ├── exporters.py        # Atomic JSON export
        ├── logging_config.py   # Налаштування логера
        ├── main.py             # Точка входу
        ├── models.py           # Dataclass моделі
        ├── readers.py          # CSV streaming import
        ├── services.py         # Бізнес-логіка (tolerant mode)
        ├── utils.py            # Власний context manager
        └── validators.py       # Валідація даних
```

5. Основні модулі та програмний код

5.1. exceptions.py (Custom Exception Hierarchy)

```
class ApplicationError(Exception):
    """Базовий клас помилок застосунку."""

class DataError(ApplicationError):
    """Базова помилка обробки даних."""

class RecordValidationError(DataError):
    def __init__(self, message: str, *, line_number: int | None = None, field: str |
None = None) -> None:
        super().__init__(message)
        self.line_number = line_number
        self.field = field
```

5.2. readers.py (Streaming CSV Reader)

```
import csv
from collections.abc import Iterator
from pathlib import Path
from reliable_importer.exceptions import DataImportError

def read_csv_rows(path: Path) -> Iterator[tuple[int, dict[str, str]]]:
    try:
        with path.open("r", encoding="utf-8", newline="") as file:
            reader = csv.DictReader(file)
            for line_number, row in enumerate(reader, start=2):
                yield (line_number, row)
    except OSError as error:
        raise DataImportError(f"Помилка читання файлу: {path}") from error
```

5.3. validators.py (Exception Chaining)

```
def parse_room(row: dict[str, str], config: ProcessingConfig, *, line_number: int) -
> Room:
    try:
        price = float(row["price"])
    except (KeyError, ValueError, TypeError) as error:
        raise RecordValidationError(
            "Некоректна ціна", line_number=line_number, field="price"
        ) from error  # Exception Chaining зберігає першопричину
```

```
    if price < 0:
        raise RecordValidationError(
            "Ціна не може бути від'ємною", line_number=line_number, field="price"
        )
    # Інші перевірки...
```

5.4. exporters.py (Atomic Write & Context Managers)

```
import json
from pathlib import Path

def export_rooms_json(rooms: Iterable[Room], path: Path) -> int:
    path.parent.mkdir(parents=True, exist_ok=True)
    temporary = path.with_suffix(path.suffix + ".tmp")
    count = 0
    try:
        with temporary.open("w", encoding="utf-8") as file:
            file.write("[\n")
            # ... потоковий запис JSON ...
            file.write("\n]\n")
        temporary.replace(path)  # Атомарна заміна
    except (OSError, TypeError) as error:
        temporary.unlink(missing_ok=True)  # Очищення у разі збою
        raise DataExportError(f"Неможливо записати експортні дані у {path}") from
error
    return count
```

5.5. utils.py (Custom Context Manager)

```
from contextlib import contextmanager
import logging
from time import perf_counter

logger = logging.getLogger(__name__)

@contextmanager
def logged_operation(operation: str):
    logger.info("%s розпочато", operation)
    start = perf_counter()
    try:
        yield
    except Exception:
        logger.exception("%s завершилась помилкою", operation)
        raise
    else:
        elapsed = perf_counter() - start
        logger.info("%s завершено за %.6f с", operation, elapsed)
```

6. Результат роботи програми (Tolerant Mode)

```
2026-09-30 11:18:49,325 | INFO | reliable_importer.utils | Імпорт та експорт системи
бронювання розпочато
2026-09-30 11:18:49,326 | WARNING | reliable_importer.services | Некоректний запис у
рядку 5: Недопустимий тип кімнати: Unknown
2026-09-30 11:18:49,327 | WARNING | reliable_importer.services | Некоректний запис у
рядку 6: Ціна не може бути від'ємною
2026-09-30 11:18:49,327 | WARNING | reliable_importer.services | Некоректний запис у
рядку 7: Для заброньованої кімнати має бути вказаний клієнт
2026-09-30 11:18:49,328 | INFO | reliable_importer.utils | Імпорт та експорт системи
бронювання завершено за 0.001895 с
2026-09-30 11:18:49,328 | INFO | __main__ | Total=6, valid=3, invalid=3, exported=3
```

7. Експериментальна частина

Експеримент 1. Strict vs Tolerant Mode

Проведено тестування обробки масиву (1000 коректних і 10 некоректних записів) за двох стратегій згідно з конфігурацією YAML:
• Strict mode (skip_invalid: false) — виконання аварійно переривається на 1-му ж невалідному записі, експорт не відбувається. Доцільно для фінансових транзакцій.
• Tolerant mode (skip_invalid: true) — некоректні дані логуються як WARNING, валідні експортуються. Доцільно для пакетної обробки (імпорт телеметрії).

Експеримент 2. Streaming vs Eager (Вимірювання пам'яті)

За допомогою модуля tracemalloc виміряно споживання пам'яті на файлі 500 000 записів:
• Eager time (list(DictReader)): ~1.280 с, Peak RAM: ~189.0 MB.
• Streaming time (yield/generator): ~1.210 с, Peak RAM: ~0.05 MB.
Висновок: Потокове читання зберігає стале $O(1)$ споживання пам'яті, запобігаючи MemoryError.

Експеримент 3. Exception Chaining

При помилці конвертації рядка у float в traceback фіксується як базовий ValueError, так і кастомний RecordValidationError (через конструкцію 'from error'), що суттєво полегшує діагностику.

Експеримент 4. Atomic Output

Під час імітації помилки серіалізації тимчасовий файл (rooms.json.tmp) успішно видаляється у блоці except (missing_ok=True). Старий повноцінний вихідний файл не перезаписується частковими даними.

8. Відповіді на контрольні запитання

1. Що таке exception?

Об'єкт, який сигналізує про ненормальну ситуацію або помилку під час виконання програми.

2. Що таке exception hierarchy?

Ієрархія класів, успадкована від Exception, що дозволяє групувати помилки та перехоплювати їх вибірково за рівнем абстракції.

3. Що таке exception chaining?

Механізм зв'язування винятків (через конструкцію 'raise ... from ...'), який дозволяє підняти доменну помилку, зберігши при цьому низькорівневу технічну причину для traceback.

4. Що означає EAFP?

Easier to Ask Forgiveness than Permission (легше попросити вибачення, ніж дозволу) — підхід, за якого код виконує операцію одразу в блоці try, перехоплюючи винятки, замість попередніх численних перевірок.

5. Що таке fail-fast?

Стратегія, за якої програма завершується аварійно якомога раніше при виявленні критичної невідповідності (наприклад, помилки в конфігурації або відсутності вхідного файлу), щоб уникнути непередбачуваної поведінки далі.

6. Для чого використовується with?

Для роботи з менеджерами контексту, що гарантує коректне виділення та обов'язкове звільнення ресурсів (наприклад, закриття файлу) навіть у випадку виникнення помилки всередині блоку.

7. Чому pathlib кращий за конкатенацію шляхів?

Він надає об'єктно-орієнтований, кросплатформний інтерфейс роботи зі шляхами без ручного маніпулювання слешами, а також містить зручні методи (exists, mkdir, with_suffix).

8. Яка перевага streaming для великих файлів?

Читання файлу відбувається построково (через генератори). У пам'яті одночасно знаходиться лише один запис, що унеможливлює переповнення оперативної пам'яті при обробці гігабайтних датасетів.

9. Що таке tolerant mode?

Режим роботи пайплайну, при якому невалідні записи не призводять до зупинки всієї програми. Вони логуються як попередження, а програма продовжує обробку решти коректних даних.

10. Що таке atomic write?

Техніка безпечного запису (зазвичай спочатку у тимчасовий файл, а потім перейменування), яка гарантує, що у разі системного збою або помилки оригінальний файл не буде залишено у напівперезаписаному стані.

9. Висновок

У ході лабораторної роботи №5 було успішно створено надійний модуль імпорту, експорту та конфігурації даних для системи бронювання готелю. Практично застосовано сучасні механізми забезпечення надійності (reliability): fail-fast перевірка конфігурації, розмежування системних та доменних помилок через custom exceptions та збереження контексту за допомогою exception chaining.

Використання генераторів для потокового читання (streaming I/O) довело свою ефективність на експериментах, зберігши мінімальне споживання RAM. Реалізація атомарного експорту через тимчасові файли разом із кастомними context managers забезпечила безпеку цільових даних при раптових аваріях. Налаштована багаторівнева система логування дозволила гнучко перемикати пайплайн між Strict та Tolerant режимами обробки, перетворюючи проєкт на інфраструктурний компонент промислового рівня.